# VTX1 Assembler Documentation

# Table of Contents


1. Authors . . . 1
2. Introduction . . . 2
3. Assembler Components . . . 3
4. VTX1 Assembler Architecture . . . 4
    4.1. Overview . . . 4
    4.2. Architecture Components . . . 4
        4.2.1. 1. Lexer (Tokenization) . . . 4
        4.2.2. 2. Parser . . . 4
        4.2.3. 3. Symbol Table Manager . . . 4
        4.2.4. 4. Code Generator . . . 4
        4.2.5. 5. Error Handler . . . 5
        4.2.6. 6. Output Formatter . . . 5
    4.3. Processing Flow . . . 5
        4.3.1. Module Responsibilities . . . 5
    4.4. Implementation Choices . . . 6
        4.4.1. Programming Language - GoLang . . . 6
        4.4.2. Error Handling Strategy . . . 6
        4.4.3. VLIW Handling . . . 6
    4.5. Module Interfaces . . . 6
        4.5.1. Lexer/Parser (ANTLR) . . . 6
        4.5.2. Parser → Codegen . . . 6
        4.5.3. Symbol Table . . . 7
        4.5.4. Error Handling . . . 7
5. Assembly Language . . . 8
6. VTX1 Assembly Language Instruction Syntax . . . 9
    6.1. Overview . . . 9
    6.2. Basic Instruction Format . . . 9
    6.3. VLIW Instruction Format . . . 9
    6.4. Register Naming . . . 9
    6.5. Addressing Modes . . . 10
    6.6. Literal Formats . . . 10
    6.7. Operation Categories . . . 11
    6.8. Example Assembly Code . . . 11
7. Instruction Encoding . . . 12
8. VTX1 Trit-Native Instruction Encoding . . . 13
    8.1. Overview . . . 13
    8.2. Physical representation . . . 13
    8.3. Operation word layout . . . 13
    8.4. Addressing . . . 14
    8.5. Registers . . . 14
    8.6. VLIW packing rules . . . 14
    8.7. Assembler representation . . . 14
    8.8. Example . . . 15

9. Appendix . . . . . 16
9.1. References . . . . . 16
9.2. Version History . . . . . 16

# Chapter 1. Authors

K. Vanyushov (itworks99)
Lead Architect and Maintainer of the VTX1 Project
GitHub: https://github.com/itworks99
Email: kvanyushov@gmail.com

# Chapter 2. Introduction

This document provides comprehensive documentation for the VTX1 assembler, which is designed to convert assembly language code into binary machine code for the VTX1 ternary processor.

The VTX1 assembler supports the complete instruction set of the VTX1 architecture, including VLIW operations with parallel execution capabilities, balanced ternary numeric representation, and various addressing modes.

# Chapter 3. Assembler Components

# Chapter 4. VTX1 Assembler Architecture

## 4.1. Overview

The VTX1 assembler is designed as a multi-pass assembler to convert VTX1 assembly language into binary machine code for the VTX1 ternary processor. This document outlines the architecture and design principles of the assembler.

## 4.2. Architecture Components

### 4.2.1. 1. Lexer (Tokenization)

The lexer processes the input source code and converts it into a stream of tokens. Each token represents a meaningful element in the assembly language such as:

- Mnemonics (e.g., ADD, SUB, LD)
- Registers (e.g., T0, TA, VA)
- Literals (decimal, binary, ternary)
- Labels and symbols
- Directives and special characters

### 4.2.2. 2. Parser

The parser processes the token stream and builds an abstract syntax tree (AST) that represents the structure of the program. It handles:

- Instruction recognition
- Operand validation
- VLIW grouping for parallel operations
- Directive processing
- Label definition and usage

### 4.2.3. 3. Symbol Table Manager

Manages symbol resolution including:

- Label definitions and references
- Global and local symbols
- Address calculation
- Forward references

### 4.2.4. 4. Code Generator

Converts the parsed instructions into binary code according to the VTX1 instruction encoding:

- 108-bit VLIW bundles (3 operation words of 18 trits each)
- 36-bit ternary word encoding

- Balanced ternary literal encoding
- Address resolution

#### 4.2.5. 5. Error Handler

Provides comprehensive error detection and reporting:

- Syntax errors
- Semantic errors (e.g., invalid register usage)
- Symbol resolution errors
- Range and value validation

#### 4.2.6. 6. Output Formatter

Generates the final output in the required format:

- Binary file format
- Hexadecimal listing
- Debug information
- Symbol table output

## 4.3. Processing Flow

The assembler operates in the following stages:

1. **Lexical Analysis (ANTLR Lexer)**: Converts source code into tokens using the ANTLR-generated lexer based on `vtx1_grammar.g4`.
2. **Parsing (ANTLR Parser)**: Parses the token stream into a parse tree using the ANTLR-generated parser. Handles instruction recognition, operand validation, VLIW grouping, and directive processing.
3. **Symbol Table Management**: Tracks label definitions, references, and symbol resolution, including forward references and address calculation.
4. **Code Generation**: Converts the parse tree (or AST) into binary machine code, handling VLIW packing, ternary encoding, and directive processing.
5. **Error Handling**: Collects and reports errors and warnings at each stage, providing line/column/source context.
6. **Output Formatting**: Generates binary, hex, or objdump output, as well as optional listing and debug information.

### 4.3.1. Module Responsibilities

- `cmd/`: Orchestrates the assembly process, CLI, and high-level flow.
- `grammar/`: Contains ANTLR grammar and generated lexer/parser code.
- `internal/` (future): Will contain symbol table, codegen, and supporting logic.
- `test/`, `examples/`: Contain test cases and sample programs.
- `docs/`: Contains architecture, syntax, and implementation documentation.

## 4.4. Implementation Choices

### 4.4.1. Programming Language - GoLang

Key considerations:

- Excellent string processing capabilities
- Rich library ecosystem
- Easy to extend and modify
- Good performance for an assembler of this scale
- Cross-platform compatibility

### 4.4.2. Error Handling Strategy

The assembler will use a comprehensive error handling strategy:

- Continue parsing after errors when possible
- Collect multiple errors before halting
- Provide clear error messages with line/column information
- Suggest potential fixes when appropriate

### 4.4.3. VLIW Handling

VLIW instructions (multiple operations per instruction word) will be handled through:

- Explicit grouping syntax in the assembly language
- Automatic validation of illegal operation combinations
- Resource conflict detection
- Pipeline hazard warnings

## 4.5. Module Interfaces

The assembler is structured with clear interfaces between its major components:

### 4.5.1. Lexer/Parser (ANTLR)

- **Input:** Source code string
- **Output:** ANTLR parse tree (`antlr.ParseTree`)
- **Interface:** ANTLR-generated Go interfaces/classes

### 4.5.2. Parser → Codegen

- **Input:** ANTLR parse tree (and/or custom AST)
- **Output:** Calls to codegen functions to emit binary
- **Interface:** Go interface for AST traversal and code emission

### 4.5.3. Symbol Table

- **Input:** Label definitions, references, and directives from parser
- **Output:** Resolved addresses and symbol errors
- **Interface:** Go struct with lookup, insert, and resolve methods

### 4.5.4. Error Handling

- **Input:** Errors from lexer, parser, codegen, and symbol table
- **Output:** Aggregated error/warning list with context
- **Interface:** Go struct with methods for add, summarize, and report

# Chapter 5. Assembly Language

# Chapter 6. VTX1 Assembly Language Instruction Syntax

## 6.1. Overview

This document describes the syntax for VTX1 assembly language instructions. The VTX1 assembler supports a sophisticated instruction set with VLIW (Very Long Instruction Word) capabilities that allow for parallel execution of multiple operations.

## 6.2. Basic Instruction Format

Individual instructions follow this general format:

```
[label:] mnemonic [operand1][, operand2][, operand3] [; comment]
```

Where:

- `label` (optional): A symbol used to reference a memory location
- `mnemonic`: The instruction name (e.g., ADD, LD, JMP)
- `operand1, operand2, operand3` (optional, instruction-dependent): Registers, literals, or symbols
- `comment` (optional): Text following a semicolon, ignored by the assembler

## 6.3. VLIW Instruction Format

The VTX1 supports VLIW instructions with up to 3 operations executed in parallel. VLIW instructions use the following syntax:

```
[op1] [op2] [op3]
```

Where each operation follows the basic instruction format. Operations are enclosed in square brackets and separated by spaces.

Example:

```
[ADD T0, T1, T2] [LD T3, [TB+1]] [NOP]
```

## 6.4. Register Naming

The VTX1 supports several register types:

1. **General Purpose Registers (GPRs)**: T0-T6
2. **Special Registers**:
   - TA: Accumulator
   - TB: Stack/Base Pointer (the PUSH/POP stack ABI)

- TC: Program Counter
- TS: Status Register
- TI: Instruction Register

3. **Vector Registers**: VA, VT, VB
4. **Floating-Point Registers**: FA, FT, FB

## 6.5. Addressing Modes

The VTX1 supports the following addressing modes:

1. **Register Direct**: The operand is a register

```
ADD T0, T1, T2  ; T0 = T1 + T2
```

2. **Immediate**: The operand is a constant value

```
ADD T0, T1, 5   ; T0 = T1 + 5
```

3. **Base + Offset**: The operand is a memory location specified by a base register plus offset

```
LD T0, [TB+4]   ; T0 = Memory[TB+4]
```

4. **Program Counter Relative**: The operand is an address relative to the current program counter

```
JMP label       ; PC = label_address
```

5. **Vector Indexed**: The operand is a vector register with an index

```
VLD VA, [TB+T0] ; VA = Memory[TB+T0]
```

## 6.6. Literal Formats

The VTX1 supports multiple literal formats:

1. **Decimal**: Standard decimal notation

```
ADD T0, T1, 42  ; Decimal 42
```

2. **Hexadecimal**: Prefixed with `0x`

```
ADD T0, T1, 0x2A  ; Hexadecimal 2A (42 decimal)
```

3. **Binary**: Prefixed with `0b`

```
ADD T0, T1, 0b101010  ; Binary 101010 (42 decimal)
```

4. **Ternary**: Prefixed with `0t` (balanced ternary notation using `-`, `0`, `+`)

```
ADD T0, T1, 0t+--+0  ; Balanced ternary +--+0 (42 decimal)
```

## 6.7. Operation Categories

The VTX1 instruction set is organized into the following categories:

1. **ALU Operations**: ADD, SUB, MUL, AND, OR, NOT, etc.
2. **Memory Operations**: LD, ST, VLD, VST, FLD, FST, etc.
3. **Control Operations**: JMP, JAL, BEQ, BNE, CALL, RET, etc.
4. **Vector Operations**: VADD, VSUB, VMUL, VAND, VOR, etc.
5. **Floating-Point Operations**: FADD, FSUB, FMUL, etc.
6. **System Operations**: NOP, WFI, etc.

## 6.8. Example Assembly Code

```
; Simple VTX1 assembly program example
        LD T0, 0x1000        ; Load value from address 0x1000 into T0
        LD T1, 0x1004        ; Load value from address 0x1004 into T1
loop:   ADD T2, T0, T1        ; T2 = T0 + T1
        [ADD T0, T1, 0] [SUB T1, T2, T0] [NOP]  ; VLIW instruction with 3 operations
        BNE T0, 0, loop       ; Branch to loop if T0 != 0
        ST T2, 0x1008        ; Store result to address 0x1008
        WFI                  ; Wait for interrupt
```

# Chapter 7. Instruction Encoding

# Chapter 8. VTX1 Trit-Native Instruction Encoding

## 8.1. Overview

VTX1 is a balanced-ternary VLIW machine. Every operation is encoded in one 18-trit machine word; a VLIW instruction is a bundle of 3 operation words (54 trits = 108 bits in the 2-bit-per-trit hardware encoding, matching `VLIW_WIDTH`).

The single source of truth for this encoding is `src/assembler/isa/isa.go`. The Verilog constants in `src/common/vtx1_isa_generated.vh` are generated from it (`cd src/assembler && go run ./isa/gen`) so the toolchain and the RTL cannot drift apart.

## 8.2. Physical representation

Each trit is stored as 2 bits (hardware encoding, see `src/common/ternary_constants.v`):

| Trit | Encoding | Meaning |
|---|---|---|
| -1 | `00` | negative |
| 0 | `01` | zero |
| +1 | `10` | positive |
| - | `11` | undefined/invalid |

An 18-trit word therefore occupies 36 bits, stored in files as 5 bytes with 4 zero padding bits. Trit `t` (0 = most significant) occupies bits `[35-2t : 34-2t]` of the word.

## 8.3. Operation word layout

All formats start with a 4-trit opcode (trits 0..3). The opcode space holds 81 values (3^4): 69 are assigned and 12 are reserved for future operations (the retired transcendental and unsigned opcodes; see `isa.ReservedValues`). The all-zero-trit word is NOP. There is no microcode: operations are either single-cycle hardwired control or dedicated multi-cycle units (DIV/MOD/SQRT, vector reductions), which stall the pipeline until done.

| Format | Layout (trit positions, MSB first) |
|---|---|
| R3 | rd 4..6 \| rs1 7..9 \| mode 10 \| payload 11..17<br>mode 0: rs2 = trits 15..17; mode +1: 7-trit immediate (±1093) |
| R2 | rd 4..6 \| rs 7..9 |
| R1 | r 4..6 |
| None | (all operand trits zero) |
| Mem | reg 4..6 \| mode 7 \| payload 8..17<br>mode 0: 10-trit direct address/immediate (±29524)<br>mode +1: base 8..10, index 11..13 (`[base]` uses index TZ)<br>mode -1: base 8..10, 7-trit offset 11..17 |
| Branch | rs1 4..6 \| rs2 7..9 \| 8-trit PC-relative word offset 10..17 (±3280); rs2 = TZ encodes "compare against zero" |
| Jump | 14-trit absolute word address, trits 4..17 (±2391484) |

Because every numeric field is balanced ternary, immediates, offsets and branch displacements are naturally signed and symmetric around zero—there is no sign bit and no two's-complement asymmetry.

Negative absolute addresses are architecturally legal and reserved for memory-mapped I/O (ratified mapping: peripheral P register k lives at word address -(P*0x10000 + k); P = 0 GPIO, 1 UART, 2 SPI, 3 I2C, 4 TIMER, 5 FLASH, 6 TMAC ternary-weight MAC accelerator). Shift and rotate amounts are the value of the low 4 trits of the second source operand (range ±40), reduced modulo 18; negative amounts shift the opposite direction.

## 8.4. Addressing

Memory is word-addressed: one address names one 18-trit word. A scalar instruction advances the PC by 1, a VLIW bundle by 3. Data directives (.DB/.DW/.DT) emit one word per item; strings emit one word per character.

## 8.5. Registers

Registers live in a unified 3-trit index space (pattern = balanced(index-13)):

| Registers | Indices | Role |
|---|---|---|
| T0-T6 | 0-6 | general purpose |
| TB | 7 | stack/base pointer (the PUSH/POP stack ABI: full-descending, word-addressed; PUSH r: TB ← TB-1, mem[TB] ← r; POP r: r ← mem[TB], TB ← TB+1; for POP TB the loaded value wins) |
| TA | 8 | accumulator |
| TC | 9 | program counter |
| TS | 10 | status |
| TI | 11 | instruction |
| VA, VT, VB | 12-14 | vector |
| FA, FT, FB | 15-17 | floating point |
| TZ | 18 | zero register: reads 0, writes discarded |

## 8.6. VLIW packing rules

The assembler enforces, per bundle:

1. Maximum of 3 operations (shorter bundles are NOP-padded)
2. Maximum of 1 memory operation
3. Maximum of 1 control operation
4. No two operations may write the same register (error)
5. An operation reading a register that another slot writes sees the pre-bundle value; the assembler emits a hazard warning

## 8.7. Assembler representation

```
[ADD T0, T0, 1] [ADD T1, T1, 1] [ADD T2, T2, 1]
```

## 8.8. Example

`BEQ T3, 0, done` four words before `done` encodes as: opcode BEQ (value 30, pattern `0-0-`), rs1 = T3 (`-0-`), rs2 = TZ (`+--`, compare against zero), offset +4 (`000000`) -- in trits: `0-0--0-+--000000`.

# Chapter 9. Appendix

## 9.1. References

- VTX1 CPU Architecture Documentation
- VTX1 Instruction Set Reference
- Balanced Ternary Logic Implementation Guide

## 9.2. Version History

| Version | Date | Changes |
|---|---|---|
| 0.1.0 | 2025-06-16 | Initial documentation structure |